分治法综合题题库（理论分析型）

题型说明

每道题的作答要求如下：

1. **分析**：说明该问题是否满足分治法的适用条件
2. **步骤**：写出分治求解的三个步骤（分解、解决、合并）
3. **伪代码**：用类C/类Pascal风格写出核心算法
4. **复杂度分析**：建立递推式 → 用递归树/主定理求解 → 给出渐进复杂度

题目1：归并排序（经典基础题）

【问题描述】 给定n个元素的序列A[1..n]，请设计分治算法对其进行升序排序。

【答题框架】

1. 分治策略分析

分治适用性判断：

- ✅ 问题可分解为规模较小的相同问题
- ✅ 子问题相互独立
- ✅ 子问题的解可合并为原问题的解

三个步骤：

- **分解**：将序列从中间位置分成两个长度相等的子序列
- **解决**：递归地对两个子序列进行归并排序
- **合并**：将两个已排序的子序列合并成一个有序序列

2. 伪代码

text

```
算法 MergeSort(A, low, high)
// 输入：待排序数组A[low..high]
// 输出：升序排列的数组A[low..high]

1.  if low < high then
2.      mid ← ⌊(low + high)/2⌋
3.      MergeSort(A, low, mid)      // 递归排序左半
4.      MergeSort(A, mid+1, high)   // 递归排序右半
```

```
5.      Merge(A, low, mid, high)    // 合并两个有序子序列
6.  end if
```

text

```
算法 Merge(A, low, mid, high)
// 合并两个有序子序列A[low..mid]和A[mid+1..high]

1.  i ← low, j ← mid+1, k ← 0
2.  B[0..high-low] ← 新数组            // 临时存储
3.  while i ≤ mid and j ≤ high do
4.      if A[i] ≤ A[j] then
5.          B[k++] ← A[i++]
6.      else
7.          B[k++] ← A[j++]
8.      end if
9.  end while
10. while i ≤ mid do                  // 处理剩余左半
11.     B[k++] ← A[i++]
12. end while
13. while j ≤ high do                 // 处理剩余右半
14.     B[k++] ← A[j++]
15. end while
16. for t ← 0 to high-low do          // 复制回原数组
17.     A[low + t] ← B[t]
18. end for
```

3. 时间复杂度分析

建立递推式:
设 T(n) 表示对n个元素排序所需时间。

- 分解：O(1)（仅计算中点）
- 解决：2T(n/2)（递归排序两个子问题）
- 合并：O(n)（Merge过程）

因此递推式为:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

方法一：递归树法

text

```
第0层:                  T(n)
                      /    \
第1层:              T(n/2)  T(n/2)       → 2 × T(n/2)
```

/ \ / \
第2层: T(n/4)T(n/4)T(n/4)T(n/4) → 4 × T(n/4)
...
第k层: 2^k × T(n/2^k)

合并代价（每层的合并工作）：
第0层: cn
第1层: 2 × c(n/2) = cn
第2层: 4 × c(n/4) = cn
...
第$\log_2 n$层: cn

总共有 $\log_2 n + 1$ 层，每层代价 cn
∴ $T(n) = cn \times (\log_2 n + 1) = O(n \log n)$

方法二：主定理法

递推式 $T(n) = 2T(n/2) + \Theta(n)$ 符合主定理标准形式：

- a = 2, b = 2, f(n) = n
- 计算 $\log_b a = \log_2 2 = 1$
- 比较 $f(n) = n$ 与 $n^{\log_b a} = n^1$

由于 $f(n) = \Theta(n^{\log_b a})$，属于主定理**情况2**

$\therefore T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n \log n)$

结论：归并排序时间复杂度为 O(n log n)

题目2：二分搜索（减治法）

【问题描述】 给定一个已按升序排列的数组A[1..n]和一个元素x，设计算法判断x是否在数组中，若存在则返回其下标。

【答题框架】

1. 分治策略分析

三个步骤：

- **分解**：取数组中间元素A[mid]与x比较
- **解决**：若相等则找到；若x < A[mid]则在左半递归查找；若x > A[mid]则在右半递归查找
- **合并**：无需合并（直接返回结果）

注：这是分治法的特例——减治法，每次问题规模减半

2. 伪代码

text

```
算法 BinarySearch(A, low, high, x)
// 输入：有序数组A[low..high]，待查找元素x
// 输出：若找到返回下标，否则返回-1

1.  if low > high then
2.      return -1
3.  end if
4.  mid ← ⌊(low + high)/2⌋
5.  if x = A[mid] then
6.      return mid
7.  else if x < A[mid] then
8.      return BinarySearch(A, low, mid-1, x)
9.  else
10.     return BinarySearch(A, mid+1, high, x)
11. end if
```

3. 时间复杂度分析

建立递推式：
每次递归只处理一个子问题（规模为原问题的一半），基本操作次数为O(1)。

$$T(n) = \begin{cases} O(1) & n = 1 \\ T(n/2) + O(1) & n > 1 \end{cases}$$

迭代展开：

$$T(n) = T(n/2) + 1$$
$$= [T(n/4) + 1] + 1 = T(n/4) + 2$$

$= T(n/8) + 3$
$= \cdots$
$= T(n/2^k) + k$

当 $n = 2^k$ 时，$T(n) = T(1) + k = 1 + \log_2 n$

递归树法：

```
text

T(n) = T(n/2) + 1
     = T(n/4) + 1 + 1
     = T(n/8) + 1 + 1 + 1
     ...
每条路径深度 log₂n，每层代价1
∴ T(n) = O(log n)
```

主定理法：
递推式 $T(n) = T(n/2) + O(1)$

- a = 1, b = 2, f(n) = 1
- $\log_b a = \log_2 1 = 0$
- $n^{\log_b a} = n^0 = 1$
- $f(n) = 1 = \Theta(1)$

属于主定理**情况2**，$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$

结论：二分搜索时间复杂度为 O(log n)

题目3：最大子段和问题

【问题描述】 给定由n个整数（可能有负数）组成的序列A[1..n]，求该序列形如 $\sum_{k=i}^{j} A[k]$ 的最大值。当所有整数均为负数时，定义最大子段和为0。

【答题框架】

1. 分治策略分析

三个步骤：

- **分解**：将序列从中间分成左右两个长度相等的子序列
- **解决**：递归求解左半最大子段和、右半最大子段和
- **合并**：计算跨越中点的最大子段和（从中点向左扩展找最大和 + 从中点向右扩展找最大和），然后取三者最大值

2. 伪代码

```
text
```

```
算法 MaxSubSum(A, low, high)
// 输入：数组A[low..high]
// 输出：最大子段和

1.  if low = high then
2.      if A[low] > 0 then return A[low]
3.      else return 0
4.  end if
5.
6.  mid ← ⌊(low + high)/2⌋
7.
8.  leftSum ← MaxSubSum(A, low, mid)          // 左半最大子段和
9.  rightSum ← MaxSubSum(A, mid+1, high)      // 右半最大子段和
10.
11. // 计算跨中点的最大子段和
12. s1 ← 0, leftPart ← 0
13. for i ← mid downto low do
14.     leftPart ← leftPart + A[i]
15.     if leftPart > s1 then s1 ← leftPart
16. end for
17.
18. s2 ← 0, rightPart ← 0
19. for i ← mid+1 to high do
20.     rightPart ← rightPart + A[i]
21.     if rightPart > s2 then s2 ← rightPart
22. end for
23.
24. crossSum ← s1 + s2
25.
26. return max(leftSum, rightSum, crossSum)
```

3. 时间复杂度分析

建立递推式：

- 分解：O(1)
- 解决：2T(n/2)（递归求解左右子问题）
- 合并：两个for循环各扫描n/2个元素 → O(n)

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

求解：
递推式与归并排序完全相同，故
$T(n) = O(n \log n)$

验证：存在更优的动态规划解法（O(n)），但分治法在理解问题结构上具有教学意义。

题目4：快速排序

【问题描述】 设计分治算法对n个元素进行排序，要求使用快速排序策略。

【答题框架】

1. 分治策略分析

三个步骤：

- **分解**：选取基准元素（如选第一个），将数组划分为两部分——左半所有元素 ≤ 基准，右半所有元素 ≥ 基准，基准处于最终位置
- **解决**：递归地对左右两个子序列进行快速排序
- **合并**：无需合并（基准已在正确位置，左右子序列已分别有序）

2. 伪代码

text

```
算法 QuickSort(A, low, high)
// 输入：数组A[low..high]
// 输出：升序排列的数组A[low..high]

1.  if low < high then
2.      pivotIndex ← Partition(A, low, high)
3.      QuickSort(A, low, pivotIndex-1)
4.      QuickSort(A, pivotIndex+1, high)
5.  end if
```

text

```
算法 Partition(A, low, high)
// 划分函数，返回基准元素的最终位置

1.  pivot ← A[low]           // 取第一个元素为基准
2.  i ← low, j ← high
3.  while i < j do
4.      while i < j and A[j] ≥ pivot do
5.          j ← j - 1
6.      end while
7.      if i < j then
8.          A[i] ← A[j]
9.          i ← i + 1
10.     end if
11.
12.     while i < j and A[i] ≤ pivot do
13.         i ← i + 1
14.     end while
15.     if i < j then
16.         A[j] ← A[i]
17.         j ← j - 1
18.     end if
19. end while
```

20. A[i] ← pivot
21. return i

3. 时间复杂度分析

最坏情况分析:
当输入序列已有序或逆序时，每次划分的基准元素总是最小（或最大）元素。

- 划分后，一个子问题规模为0，另一个规模为n-1

递推式:
$$T(n) = T(n-1) + \Theta(n)$$

展开:
$$T(n) = T(n-1) + n = T(n-2) + (n-1) + n = \cdots = T(1) + \sum_{k=2}^{n} k = \Theta(n^2)$$

最好情况分析:
每次划分都将数组均匀分成两半

递推式:
$$T(n) = 2T(n/2) + \Theta(n)$$

由主定理（a=2, b=2, f(n)=n, $\log_b a = 1$）:
$$T(n) = \Theta(n \log n)$$

平均情况分析:
假设每次划分基准元素的最终位置等概率出现在1~n的任何位置

$$T(n) = \frac{1}{n} \sum_{k=1}^{n} [T(k-1) + T(n-k) + \Theta(n)]$$

可以证明平均时间复杂度为 $\Theta(n \log n)$

结论：快速排序最坏O(n²)，平均O(n log n)

题目5：线性时间选择（第k小元素）

【问题描述】 给定n个元素的序列和整数k（1≤k≤n），设计分治算法找出序列中第k小的元素，要求最坏情况时间复杂度为O(n)。

【答题框架】

1. 分治策略分析（BFPRT算法）

思想：改进快速排序的划分方法，通过精心选择基准来保证划分的平衡性

三个步骤:

- **分解**:
  1. 将数组分成⌈n/5⌉组，每组5个元素（最后一组可能不足5个）
  2. 对每组排序，取出各组的中位数

3. 递归调用选择算法找出这些中位数的中位数x（作为基准）
4. 用x对数组进行划分

- **解决**：根据k与划分后基准位置的关系，递归地在左半或右半中继续查找
- **合并**：无需合并

2. 伪代码

text

```
算法 Select(A, low, high, k)
// 输入：数组A[low..high]，整数k
// 输出：第k小的元素

1.  if high - low + 1 ≤ 5 then
2.      对A[low..high]排序
3.      return A[low + k - 1]
4.  end if
5.
6.  // 分组并找各组中位数
7.  for i ← 0 to ⌈(high-low+1)/5⌉-1 do
8.      对第i组的5个元素排序
9.      将第i组的中位数交换到A[low + i]
10. end for
11.
12. // 递归找中位数的中位数
13. x ← Select(A, low, low + ⌈(high-low+1)/5⌉ - 1, ⌈(high-low+1)/10⌉)
14.
15. // 用x划分数组
16. pivotIndex ← Partition(A, low, high, x)
17.
18. // 递归求解
19. if k = pivotIndex - low + 1 then
20.     return x
21. else if k < pivotIndex - low + 1 then
22.     return Select(A, low, pivotIndex-1, k)
23. else
24.     return Select(A, pivotIndex+1, high, k - (pivotIndex-low+1))
25. end if
```

3. 时间复杂度分析

建立递推式：

- 每组排序：每组5个元素，O(1)时间，共⌈n/5⌉组 → O(n)
- 找中位数的中位数：递归调用Select → T(⌈n/5⌉)
- 用x划分：O(n)
- 递归求解子问题：最多处理约7n/10个元素（可证明至少有3n/10个元素小于x，3n/10个大于x）

$$T(n) \le T(\lceil n/5 \rceil) + T(7n/10) + O(n)$$

求解：
用递归树法，每层总工作量递减：
$$T(n) \le T(\lceil n/5 \rceil) + T(7n/10) + cn$$
$$\le c(n + 9n/10 + 81n/100 + \cdots) = O(n)$$

结论：BFPRT选择算法最坏时间复杂度为 O(n)

题目6：Strassen矩阵乘法

【问题描述】 给定两个n×n矩阵A和B（n为2的幂），设计分治算法计算乘积C = A×B，要求比传统O(n³)算法更优。

【答题框架】

1. 分治策略分析

传统分治法（低效）：
将每个矩阵分成4个n/2×n/2子块：
$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

需要8次子矩阵乘法和4次子矩阵加法：
$$T(n) = 8T(n/2) + O(n^2)$$

解得 $T(n) = O(n^3)$，未改进。

Strassen改进：通过数学变换，将乘法次数从8次减少到7次

Strassen七步：

1. $M_1 = A_{11}(B_{12} - B_{22})$
2. $M_2 = (A_{11} + A_{12})B_{22}$
3. $M_3 = (A_{21} + A_{22})B_{11}$
4. $M_4 = A_{22}(B_{21} - B_{11})$
5. $M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$
6. $M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$
7. $M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$

计算结果：

- $C_{11} = M_5 + M_4 - M_2 + M_6$
- $C_{12} = M_1 + M_2$
- $C_{21} = M_3 + M_4$
- $C_{22} = M_5 + M_1 - M_3 - M_7$

2. 伪代码

text

```
算法 StrassenMultiply(A, B, n)
// 输入：两个n×n矩阵A和B（n为2的幂）
// 输出：乘积矩阵C = A×B

1.  if n = 1 then
2.      C[1][1] ← A[1][1] × B[1][1]
3.      return C
4.  end if
5.
6.  将A和B划分为4个n/2×n/2子矩阵
7.
8.  M1 ← StrassenMultiply(A11, B12 - B22, n/2)
9.  M2 ← StrassenMultiply(A11 + A12, B22, n/2)
10. M3 ← StrassenMultiply(A21 + A22, B11, n/2)
11. M4 ← StrassenMultiply(A22, B21 - B11, n/2)
12. M5 ← StrassenMultiply(A11 + A22, B11 + B22, n/2)
13. M6 ← StrassenMultiply(A12 - A22, B21 + B22, n/2)
14. M7 ← StrassenMultiply(A11 - A21, B11 + B12, n/2)
15.
16. // 合并结果
17. C11 ← M5 + M4 - M2 + M6
18. C12 ← M1 + M2
19. C21 ← M3 + M4
20. C22 ← M5 + M1 - M3 - M7
21.
22. return C
```

3. 时间复杂度分析

建立递推式：

- 需要进行7次n/2阶矩阵乘法
- 加减法运算：每次加法/减法为O(n²)

$$T(n) = \begin{cases} O(1) & n = 1 \\ 7T(n/2) + O(n^2) & n > 1 \end{cases}$$

主定理求解：

- a = 7, b = 2, f(n) = O(n²)
- $\log_b a = \log_2 7 \approx 2.807$

由于 $n^2 = O(n^{2.807-\epsilon})$，即 $f(n) = O(n^{\log_b a - \epsilon})$，属于主定理**情况1**

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

结论：Strassen算法时间复杂度 O(n^{2.807})，优于传统 O(n³)，是目前理论上较好的矩阵乘法算法之一。

附：分治法时间复杂度分析总结表

算法	递推式	log_b a	主定理情况	时间复杂度
归并排序	$T(n)=2T(n/2)+O(n)$	1	情况2	$O(n \log n)$
二分搜索	$T(n)=T(n/2)+O(1)$	0	情况2	$O(\log n)$
最大子段和	$T(n)=2T(n/2)+O(n)$	1	情况2	$O(n \log n)$
快速排序(最好)	$T(n)=2T(n/2)+O(n)$	1	情况2	$O(n \log n)$
快速排序(最坏)	$T(n)=T(n-1)+O(n)$	—	递推法	$O(n^2)$
线性时间选择	$T(n)\leq T(n/5)+T(7n/10)+O(n)$	—	递归树	$O(n)$
Strassen乘法	$T(n)=7T(n/2)+O(n^2)$	$\log_2 7\approx 2.807$	情况1	O(n^{2.807})